

6.5840 Lab 2: Key/Value Server

[Collaboration policy](#) // [Submit lab](#) // [Setup Go](#) // [Guidance](#) // [Piazza](#)

Introduction

In this lab you will build a key/value server for a single machine that ensures that each Put operation is executed *at-most-once* despite network failures and that the operations are *linearizable*. You will use this KV server to implement a lock. Later labs will replicate a server like this one to handle server crashes.

KV server

Each client interacts with the key/value server using a *Clerk*, a set of library routines which sends RPCs to the server. Clients can send two different RPCs to the server: Put(key, value, version) and Get(key). The server maintains an in-memory map that records for each key a (value, version) tuple. Keys and values are strings. The version number records the number of times the key has been written. Put(key, value, version) installs or replaces the value for a particular key in the map *only if* the Put's version number matches the server's version number for the key. If the version numbers match, the server also increments the version number of the key. If the version numbers don't match, the server should return `rpc.ErrVersion`. A client can create a new key by invoking Put with version number 0 (and the resulting version stored by the server will be 1). If the version number of the Put is larger than 0 and the key doesn't exist, the server should return `rpc.ErrNoKey`.

Get(key) fetches the current value for the key and its associated version. If the key doesn't exist at the server, the server should return `rpc.ErrNoKey`.

Maintaining a version number for each key will be useful for implementing locks using Put and ensuring at-most-once semantics for Put's when the network is unreliable and the client retransmits.

When you've finished this lab and passed all the tests, you'll have a *linearizable* key/value service from the point of view of clients calling Clerk.Get and Clerk.Put. That is, if client operations aren't concurrent, each client Clerk.Get and Clerk.Put will observe the modifications to the state implied by the preceding sequence of operations. For concurrent operations, the return values and final state will be the same as if the operations had executed one at a time in some order. Operations are concurrent if they overlap in time: for example, if client X calls Clerk.Put(), and client Y calls Clerk.Put(), and then client X's call returns. An operation must observe the effects of all operations that have completed before the operation starts. See the FAQ on [linearizability](#) for more background.

Linearizability is convenient for applications because it's the behavior you'd see from a single server that processes requests one at a time. For example, if one client gets a successful response from the server for an update request, subsequently launched reads from other clients are guaranteed to see the effects of that update. Providing linearizability is relatively easy for a single server.

Getting Started

We supply you with skeleton code and tests in `src/kvsrv1`. `kvsrv1/client.go` implements a Clerk that clients use to manage RPC interactions with the server; the Clerk provides Put and Get methods. `kvsrv1/server.go` contains the server code, including the Put and Get handlers that implement the server side of RPC requests. You will need to modify `client.go` and `server.go`. The RPC requests, replies, and error values are defined in the `kvsrv1/rpc` package in the file `kvsrv1/rpc/rpc.go`, which you should look at, though you don't have to modify `rpc.go`.

To get up and running, execute the following commands. Don't forget the git pull to get the latest software.

```
$ cd ~/6.5840
$ git pull
...
$ cd src
$ make kvsrv1
go build -race -o main/kvsrv1d main/kvsrv1d.go
cd kvsrv1 && go test -v -race
=== RUN    TestReliablePut
One client and reliable Put (reliable network)...
    kvsrv_test.go:25: Put err ErrNoKey
--- FAIL: TestReliablePut (0.31s)
...
$
```

Key/value server with reliable network

Your first task is to implement a solution that works when there are no dropped messages. You'll need to add RPC-sending code to the Clerk Put/Get methods in `client.go`, and implement Put and Get RPC handlers in `server.go`.

You have completed this task when you pass the Reliable tests in the test suite:

```
$ cd src
$ make RUN="-run Reliable" kvsrv1
go build -race -o main/kvsrv1d main/kvsrv1d.go
cd kvsrv1 && go test -v -race -run Reliable
=== RUN    TestReliablePut
One client and reliable Put (reliable network)...
    ... Passed -- time 0.0s #peers 1 #RPCs 5 #Ops 5
--- PASS: TestReliablePut (0.12s)
=== RUN    TestPutConcurrentReliable
Test: many clients racing to put values to the same key (reliable network)...
    ... Passed -- time 6.3s #peers 1 #RPCs 11025 #Ops 22050
--- PASS: TestPutConcurrentReliable (6.36s)
=== RUN    TestMemPutManyClientsReliable
Test: memory use many put clients (reliable network)...
    ... Passed -- time 29.0s #peers 1 #RPCs 50000 #Ops 50000
--- PASS: TestMemPutManyClientsReliable (52.91s)
PASS
ok      6.5840/kvsrv1    60.732s
$
```

The numbers after each Passed are real time in seconds, the constant 1, the number of RPCs sent (including client RPCs), and the number of key/value operations executed (Clerk Get and Put calls).

Implementing a lock using key/value clerk

In many distributed applications, clients running on different machines use a key/value server to coordinate their activities. For example, ZooKeeper and Etcd allow clients to coordinate using a distributed lock, in analogy with how threads in a Go program can coordinate with locks (i.e., `sync.Mutex`). Zookeeper and Etcd implement such a lock with conditional put.

Your task is to implement locks, using your key/value server to store whatever per-lock state your design needs. There can be multiple independent locks, each with its own name, passed as an argument to `MakeLock`. A lock supports two methods: `Acquire` and `Release`. The specification is that only one client can successfully acquire a given lock at a time; other clients must wait until the first client has released the lock using `Release`.

We supply you with skeleton code and tests in `src/kvsrv1/lock/`. You will need to modify `src/kvsrv1/lock/lock.go`. Your `Acquire` and `Release` should store each lock's state in your key/value server, by calling `lk.ck.Put()` and `lk.ck.Get()`.

If a client crashes while holding a lock, the lock will never be released. In a design more sophisticated than this lab, the client would attach a [lease](#) to a lock. When the lease expires, the lock server would release the lock on behalf of the client. In this lab clients don't crash and you can ignore this problem.

Implement `Acquire` and `Release`. You have completed this exercise when your code passes these tests:

```
$ cd src
$ make RUN="-run Reliable" lock1
go build -race -o main/kvsrv1d main/kvsrv1d.go
cd kvsrv1/lock; go test -v -race -run Reliable
=== RUN    TestReliableBasic
Test: a single Acquire and Release (reliable network)...
... Passed -- time 0.0s #peers 1 #RPCs 4 #Ops 4
--- PASS: TestReliableBasic (0.13s)
=== RUN    TestReliableNested
Test: one client, two locks (reliable network)...
... Passed -- time 0.1s #peers 1 #RPCs 17 #Ops 17
--- PASS: TestReliableNested (0.17s)
=== RUN    TestOneClientReliable
Test: 1 lock clients (reliable network)...
... Passed -- time 2.0s #peers 1 #RPCs 477 #Ops 477
--- PASS: TestOneClientReliable (2.14s)
=== RUN    TestManyClientsReliable
Test: 10 lock clients (reliable network)...
... Passed -- time 2.2s #peers 1 #RPCs 5704 #Ops 5704
--- PASS: TestManyClientsReliable (2.36s)
PASS
ok      6.5840/kvsrv1/lock      5.817s
$
```

If you haven't implemented the lock yet, the first two tests will succeed.

This exercise requires little code but a bit more independent thought than the previous exercise.

- You will need a unique identifier for each lock client; call `kvtest.RandValue(8)` to generate a random string.

Key/value server with dropped messages

The main challenge in this exercise is that the network may re-order, delay, or discard RPC requests and/or replies. To recover from discarded requests/replies, the Clerk must keep re-trying each RPC until it receives a reply from the server.

If the network discards an RPC request message, then the client re-sending the request will solve the problem: the server will receive and execute just the re-sent request.

However, the network might instead discard an RPC reply message. The client does not know which message was discarded; the client only observes that it received no reply. If it was the reply that was discarded, and the client re-sends the RPC request, then the server will receive two copies of the request. That's OK for a Get, since Get doesn't modify the server state. It is safe to resend a Put RPC with the same version number, since the server executes Put conditionally on the version number; if the server received and executed a Put RPC, it will respond to a re-transmitted copy of that RPC with `rpc.ErrVersion` rather than executing the Put a second time.

A tricky case is if the server replies with an `rpc.ErrVersion` in a response to an RPC that the Clerk retried. In this case, the Clerk cannot know if the Clerk's Put was executed by the server or not: the first RPC might have been executed by the server but the network may have discarded the successful response from the server, so that the server sent `rpc.ErrVersion` only for the retransmitted RPC. Or, it might be that another Clerk updated the key before the Clerk's first RPC arrived at the server, so that the server executed neither of the Clerk's RPCs and replied `rpc.ErrVersion` to both. Therefore, if a Clerk receives `rpc.ErrVersion` for a retransmitted Put RPC, `Clerk.Put` must return `rpc.ErrMaybe` to the application instead of `rpc.ErrVersion` since the request may have been executed. It is then up to the application to handle this case. If the server responds to an initial (not retransmitted) Put RPC with `rpc.ErrVersion`, then the Clerk should return `rpc.ErrVersion` to the application, since the RPC was definitely not executed by the server.

It would be more convenient for application developers if Put's were exactly-once (i.e., no `rpc.ErrMaybe` errors) but that is difficult to guarantee without maintaining state at the server for each Clerk. In the last exercise of this lab, you will implement a lock using your Clerk to explore how to program with at-most-once `Clerk.Put`.

Now you should modify your `kvsrv1/client.go` to continue in the face of dropped RPC requests and replies. A return value of `true` from the client's `ck.cnt.Call()` indicates that the client received an RPC reply from the server; a return value of `false` indicates that it did not receive a reply (more precisely, `Call()` waits for a reply message for a timeout interval, and returns `false` if no reply arrives within that time). Your Clerk should keep re-sending an RPC until it receives a reply. Keep in mind the discussion of `rpc.ErrMaybe` above. Your solution shouldn't require any changes to the server.

Add code to Clerk to retry if doesn't receive a reply. You have completed this task if your code passes all the tests for `kvsrv1`:

```
$ make kvsrv1
go build -race -o main/kvsrv1d main/kvsrv1d.go
cd kvsrv1 && go test -v -race
=== RUN    TestReliablePut
One client and reliable Put (reliable network)...
... Passed -- time 0.0s #peers 1 #RPCs      5 #Ops      5
--- PASS: TestReliablePut (0.12s)
=== RUN    TestPutConcurrentReliable
Test: many clients racing to put values to the same key (reliable network)...
```

```

... Passed -- time 6.4s #peers 1 #RPCs 11021 #Ops 22042
--- PASS: TestPutConcurrentReliable (6.52s)
=== RUN TestMemPutManyClientsReliable
Test: memory use many put clients (reliable network)...
... Passed -- time 28.8s #peers 1 #RPCs 50000 #Ops 50000
--- PASS: TestMemPutManyClientsReliable (52.44s)
=== RUN TestUnreliableNet
One client (unreliable network)...
... Passed -- time 4.0s #peers 1 #RPCs 268 #Ops 422
--- PASS: TestUnreliableNet (4.13s)
PASS
ok      6.5840/kvsrv1 64.442s
$

```

- Before the client retries, it should wait a little bit; you can use go's time package and call `time.Sleep(100 * time.Millisecond)`

Implementing a lock using key/value clerk and unreliable network

Modify your lock implementation to work correctly with your modified key/value client when the network is not reliable. You have completed this exercise when your code passes all the lock1 tests:

```

$ make lock1
go build -race -o main/kvsrvld main/kvsrvld.go
cd kvsrv1/lock; go test -v -race
=== RUN TestReliableBasic
Test: a single Acquire and Release (reliable network)...
... Passed -- time 0.0s #peers 1 #RPCs 4 #Ops 4
--- PASS: TestReliableBasic (0.13s)
=== RUN TestReliableNested
Test: one client, two locks (reliable network)...
... Passed -- time 0.1s #peers 1 #RPCs 17 #Ops 17
--- PASS: TestReliableNested (0.17s)
=== RUN TestOneClientReliable
Test: 1 lock clients (reliable network)...
... Passed -- time 2.0s #peers 1 #RPCs 471 #Ops 471
--- PASS: TestOneClientReliable (2.13s)
=== RUN TestManyClientsReliable
Test: 10 lock clients (reliable network)...
... Passed -- time 2.2s #peers 1 #RPCs 5658 #Ops 5658
--- PASS: TestManyClientsReliable (2.35s)
=== RUN TestOneClientUnreliable
Test: 1 lock clients (unreliable network)...
... Passed -- time 2.1s #peers 1 #RPCs 66 #Ops 57
--- PASS: TestOneClientUnreliable (2.18s)
=== RUN TestManyClientsUnreliable
Test: 10 lock clients (unreliable network)...
... Passed -- time 4.1s #peers 1 #RPCs 778 #Ops 617
--- PASS: TestManyClientsUnreliable (4.23s)
PASS
ok      6.5840/kvsrv1/lock 12.227s
$

```

